

Lab 8: Mutable Trees

lab08.zip (lab08.zip)

Due by 11:59pm on Wednesday, October 30.

Starter Files

Download [lab08.zip](#) (lab08.zip).

Topics

Consult this section if you need a refresher on the material for this lab. It's okay to skip directly to [the questions](#) and refer back here should you get stuck.

Mutable Trees

Mutable Trees

A `Tree` instance has two instance attributes:

- `label` is the value stored at the root of the tree.
- `branches` is a list of `Tree` instances that hold the labels in the rest of the tree.

The `Tree` class (with its `__repr__` and `__str__` methods omitted) is defined as:

```

class Tree:
    """A tree has a label and a list of branches.

    >>> t = Tree(3, [Tree(2, [Tree(5)]), Tree(4)])
    >>> t.label
    3
    >>> t.branches[0].label
    2
    >>> t.branches[1].is_leaf()
    True
    """
    def __init__(self, label, branches=[]):
        self.label = label
        for branch in branches:
            assert isinstance(branch, Tree)
        self.branches = list(branches)

    def is_leaf(self):
        return not self.branches

```

To construct a `Tree` instance from a label `x` (any value) and a list of branches `bs` (a list of `Tree` instances) and give it the name `t`, write `t = Tree(x, bs)`.

For a tree `t`:

- Its root label can be any value, and `t.label` evaluates to it.
- Its branches are always `Tree` instances, and `t.branches` evaluates to the **list** of its branches.
- `t.is_leaf()` returns `True` if `t.branches` is empty and `False` otherwise.
- To construct a leaf with label `x`, write `Tree(x)`.

Displaying a tree `t`:

- `repr(t)` returns a Python expression that evaluates to an equivalent tree.
- `str(t)` returns one line for each label indented once more than its parent with children below their parents.

```

>>> t = Tree(3, [Tree(1, [Tree(4), Tree(1)]), Tree(5, [Tree(9)])])

>>> t          # displays the contents of repr(t)
Tree(3, [Tree(1, [Tree(4), Tree(1)]), Tree(5, [Tree(9)])])

>>> print(t)   # displays the contents of str(t)
3
  1
    4
    1
  5
    9

```

Changing (also known as mutating) a tree `t` :

- `t.label = y` changes the root label of `t` to `y` (any value).
- `t.branches = ns` changes the branches of `t` to `ns` (a list of `Tree` instances).
- Mutation of `t.branches` will change `t`. For example, `t.branches.append(Tree(y))` will add a leaf labeled `y` as the right-most branch.
- Mutation of any branch in `t` will change `t`. For example, `t.branches[0].label = y` will change the root label of the left-most branch to `y`.

```
>>> t.label = 3.0
>>> t.branches[1].label = 5.0
>>> t.branches.append(Tree(2, [Tree(6)]))
>>> print(t)
3.0
  1
    4
    1
  5.0
    9
  2
    6
```

Here is a summary of the differences between the tree data abstraction implemented as a functional abstraction vs. implemented as a class:

-	Tree constructor and selector functions	Tree class
Constructing a tree	To construct a tree given a <code>label</code> and a list of <code>branches</code> , we call <code>tree(label, branches)</code>	To construct a tree object given a <code>label</code> and a list of <code>branches</code> , we call <code>Tree(label, branches)</code> (which calls the <code>Tree.__init__</code> method).
Label and branches	To get the label or branches of a tree <code>t</code> , we call <code>label(t)</code> or <code>branches(t)</code> respectively	To get the label or branches of a tree <code>t</code> , we access the instance attributes <code>t.label</code> or <code>t.branches</code> respectively.
Mutability	The functional tree data abstraction is immutable (without violating its abstraction barrier) because we cannot assign values to call expressions	The <code>label</code> and <code>branches</code> attributes of a <code>Tree</code> instance can be reassigned, mutating the tree.
Checking if a tree is a leaf	To check whether a tree <code>t</code> is a leaf, we call the function <code>is_leaf(t)</code>	To check whether a tree <code>t</code> is a leaf, we call the method <code>t.is_leaf()</code> . This method can only be called on <code>Tree</code> objects.

Required Questions

Getting Started Videos

Mutable Trees

Q1: WWPDP: Trees

Read over the `Tree` class in `lab08.py`. Make sure you understand the doctests.

Use Ok to test your knowledge with the following "What Would Python Display?" questions:

```
python3 ok -q trees-wwpd -u
```

Enter `Function` if you believe the answer is `<function ...>`, `Error` if it errors, and `Nothing` if nothing is displayed. Recall that `Tree` instances will be displayed the same way they are constructed.

```

>>> t = Tree(1, Tree(2))
-----

>>> t = Tree(1, [Tree(2)])
>>> t.label
-----

>>> t.branches[0]
-----

>>> t.branches[0].label
-----

>>> t.label = t.branches[0].label
>>> t
-----

>>> t.branches.append(Tree(4, [Tree(8)]))
>>> len(t.branches)
-----

>>> t.branches[0]
-----

>>> t.branches[1]
-----

```

Q2: Cumulative Mul

Write a function `cumulative_mul` that mutates the `Tree t` so that each node's label is replaced by the product of its label and the labels of all its descendents.

Hint: Be careful of the order in which you mutate the current node's label and process its subtrees; which one should come first?

```
def cumulative_mul(t):
    """Mutates t so that each node's label becomes the product of its own
    label and all labels in the corresponding subtree rooted at t.

    >>> t = Tree(1, [Tree(3, [Tree(5)]), Tree(7)])
    >>> cumulative_mul(t)
    >>> t
    Tree(105, [Tree(15, [Tree(5)]), Tree(7)])
    >>> otherTree = Tree(2, [Tree(1, [Tree(3), Tree(4), Tree(5)]), Tree(6, [Tree(7)])])
    >>> cumulative_mul(otherTree)
    >>> otherTree
    Tree(5040, [Tree(60, [Tree(3), Tree(4), Tree(5)]), Tree(42, [Tree(7)])])
    """
    "*** YOUR CODE HERE ***"
```

Use Ok to test your code:

```
python3 ok -q cumulative_mul
```



Q3: Prune Small

Removing some nodes from a tree is called *pruning* the tree.

Complete the function `prune_small` that takes in a `Tree t` and a number `n`. For each node with more than `n` branches, keep only the `n` branches with the smallest labels and remove (*prune*) the rest.

Hint: The `max` function takes in an iterable as well as an optional `key` argument (which takes in a one-argument function). For example, `max([-7, 2, -1], key=abs)` would return `-7` since `abs(-7)` is greater than `abs(2)` and `abs(-1)`.

```

def prune_small(t, n):
    """Prune the tree mutatively, keeping only the n branches
    of each node with the smallest labels.

    >>> t1 = Tree(6)
    >>> prune_small(t1, 2)
    >>> t1
    Tree(6)
    >>> t2 = Tree(6, [Tree(3), Tree(4)])
    >>> prune_small(t2, 1)
    >>> t2
    Tree(6, [Tree(3)])
    >>> t3 = Tree(6, [Tree(1), Tree(3, [Tree(1), Tree(2), Tree(3)]), Tree(5, [Tree(3),
    >>> prune_small(t3, 2)
    >>> t3
    Tree(6, [Tree(1), Tree(3, [Tree(1), Tree(2)])])
    """

    while ____:
        largest = max(____, key=____)
        t.branches.remove(largest)
    for b in t.branches:
        ____

```

Use Ok to test your code:

```
python3 ok -q prune_small
```



Q4: Delete

Implement `delete`, which takes a `Tree t` and removes all non-root nodes labeled `x`. The parent of each remaining node is its nearest ancestor that was not removed. The root node is never removed, even if its label is `x`.

```
def delete(t, x):
    """Remove all nodes labeled x below the root within Tree t. When a non-leaf
    node is deleted, the deleted node's children become children of its parent.

    The root node will never be removed.

    >>> t = Tree(3, [Tree(2, [Tree(2), Tree(2)]), Tree(2), Tree(2, [Tree(2, [Tree(2), 1
    >>> delete(t, 2)
    >>> t
    Tree(3)
    >>> t = Tree(1, [Tree(2, [Tree(4, [Tree(2)]), Tree(5)]), Tree(3, [Tree(6), Tree(2)]
    >>> delete(t, 2)
    >>> t
    Tree(1, [Tree(4), Tree(5), Tree(3, [Tree(6)]), Tree(4)])
    >>> t = Tree(1, [Tree(2, [Tree(4), Tree(5)]), Tree(3, [Tree(6), Tree(2)]), Tree(2,
    >>> delete(t, 2)
    >>> t
    Tree(1, [Tree(4), Tree(5), Tree(3, [Tree(6)]), Tree(6), Tree(7), Tree(8), Tree(4)])
    """
    new_branches = []
    for _____ in _____:
        _____
        if b.label == x:
            _____
        else:
            _____
    t.branches = _____
```

Use Ok to test your code:

```
python3 ok -q delete
```



Check Your Score Locally

You can locally check your score on each question of this assignment by running

```
python3 ok --score
```

This does NOT submit the assignment! When you are satisfied with your score, submit the assignment to Gradescope to receive credit for it.

Submit Assignment

If you are in a regular section of CS 61A, fill out this lab attendance and feedback form (<https://forms.gle/dHxj8gttNWRy6Ptm9>). (If you are in the mega section, you don't need to fill out the form.)

Then, submit this assignment by uploading any files you've edited **to the appropriate Gradescope assignment**. Lab 00 ([../lab00/#submit-with-gradescope](https://gradescope.com/submissions/61a-s21/lab00/#submit-with-gradescope)) has detailed instructions.

Optional Questions

Q5: Maximum Path Sum

Write a function that takes in a tree and returns the maximum sum of the values along any path from the root to a leaf of the tree.

```
def max_path_sum(t):  
    """Return the maximum path sum of the tree.  
  
    >>> t = Tree(1, [Tree(5, [Tree(1), Tree(3)]), Tree(10)])  
    >>> max_path_sum(t)  
    11  
    """  
  
    """*** YOUR CODE HERE ***"""
```

Use Ok to test your code:

```
python3 ok -q max_path_sum
```

